

BLE-Controlled RC Car Using Arduino UNO R4 WiFi

1. Project Overview

The goal of this project is to design and implement a remote-controlled (RC) vehicle using the Arduino UNO R4 WiFi board. The vehicle is controlled wirelessly via Bluetooth Low Energy (BLE) using a mobile application, and movement is achieved through two DC motors driven by an L298N dual H-bridge motor driver.

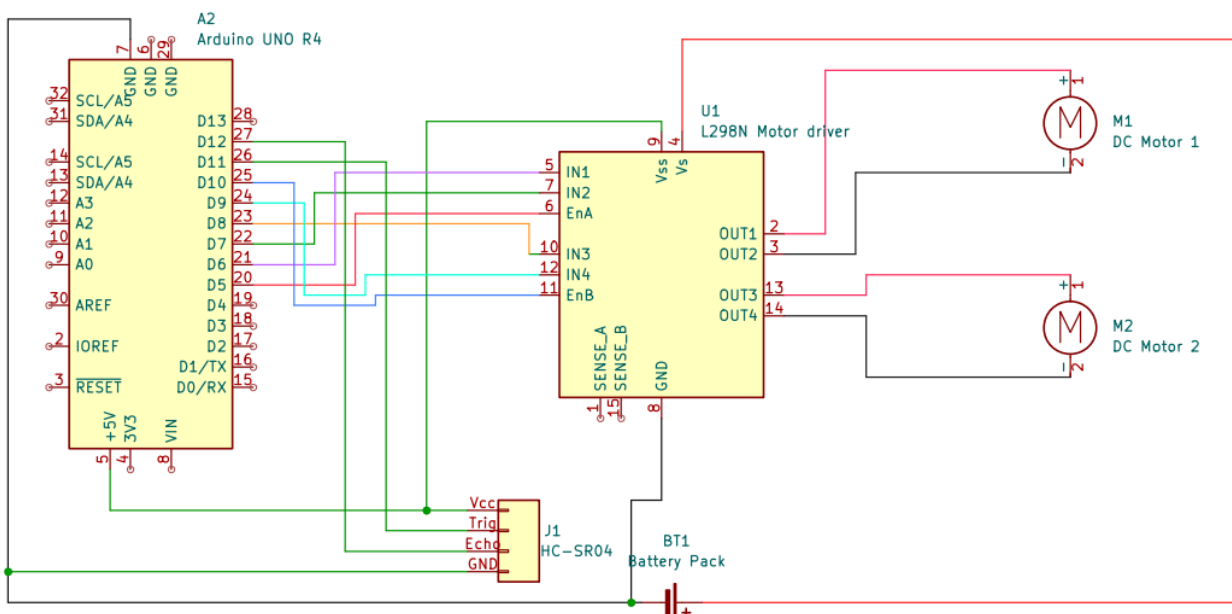
In addition to basic motion control, the project implements:

- Adjustable motor speed
- Dual driving modes: one-press and hold to move
- Visual feedback using the built-in 12x8 LED matrix
- Active Safety: HC-SR04 Ultrasonic sensor for Automatic Emergency Braking (AEB)
- Modular and extensible software architecture

2. Hardware components

- Arduino UNO R4 WiFi (with ESP32-S3-Mini-1 co-processor for WiFi and Bluetooth)
- L298N Dual H-Bridge
- HC-SR04 Ultrasonic Sensor
- 2 DC motors
- Power supply - 6 AA batteries

3. Schematic



4. Software Architecture

The software is written in C++ using the Arduino IDE and is divided into independent modules, each responsible for a specific function.

File Structure

- Main.ino
- motor_control.cpp (.h)
- ble_control.cpp (.h)
- sensor_control.cpp(.h)
- LEDmatrix_control.cpp (.h)

The modular approach improves readability and enables easier expansion.

5. Motor Control Module

The `motor_control` module abstracts the H-Bridge logic, allowing the main loop to interact with "directions" rather than specific pins.

Functions

setupMotors: sets pins in1, in2, in3, in4, ena and enb as outputs.

Direction functions: Depending on the movement direction (forward, backward, right, left, stop), using `digitalWrite` and `analogWrite` to set the appropriate value to the specific pins.

Example:

```
void turnLeft(int speed) {  
    digitalWrite(IN1, HIGH);  
    digitalWrite(IN2, LOW);  
  
    digitalWrite(IN3, LOW);  
    digitalWrite(IN4, HIGH);  
  
    analogWrite(ENA, speed);  
    analogWrite(ENB, speed);  
}
```

To turn left the right motor needs to go forward and the left one backward. IN1 and IN2 are connected to the positive and negative terminals of the right motor so setting the positive to high

and the negative to low causes forward spinning. The opposite for the left motor. ENA and ENB control the speed of the motors. Movement in other directions works similarly.

Motion Truth Table: The following logic levels are applied to the IN pins to achieve movement:

Direction	IN1	IN2	IN3	IN4	Logic Explanation
Forward	HIGH	LOW	HIGH	LOW	Both motors spin forward.
Backward	LOW	HIGH	LOW	HIGH	Polarity is reversed for both motors.
Left Turn	HIGH	LOW	LOW	HIGH	Right motor forward, Left motor backward.
Right Turn	LOW	HIGH	HIGH	LOW	Left motor forward, Right motor backward.
Stop	LOW	LOW	LOW	LOW	All coils de-energized.

6. Sensor Module

The vehicle uses "Active Sonar" to prevent collisions. The system measures the time for an ultrasonic pulse to return to determine distance.

- **AEB Logic:** If an obstacle is detected within 15 cm while moving forward, the system triggers `stopMotors()`, displays the Stop icon, and overrides user commands until the path is clear.
 - **Automatic Slowdown:** If an obstacle is detected within 40 cm, the `speedValue` is automatically capped to ensure safer reaction times.
-

7. BLE Communication Module

ArduinoBLE Library Overview

The ArduinoBLE library is Arduino's official BLE library. It lets the board act as a BLE peripheral (server). It advertises itself and provides services and characteristics that another device (a phone) can connect to and read/write.

- **Peripheral (Server):** Arduino. It exposes services and characteristics.
- **Central (client):** Phone application. It scans for peripherals and interacts with them.
- **Service:** Grouping of data
- **Characteristic:** A single variable inside a service. Can be readable, writable, or notify-capable.
- **UUID (Universally Unique Identifier):** A unique ID to identify a service or characteristic. 16-bit for standard BLE service or 128-bit for custom.

For the RC car:

- **Arduino = Peripheral**
 - **BLE service = "CarControlService"**
 - **Characteristic = "CommandCharacteristic" (phone writes "F", "B", "L", "R", "S", "T")**
-

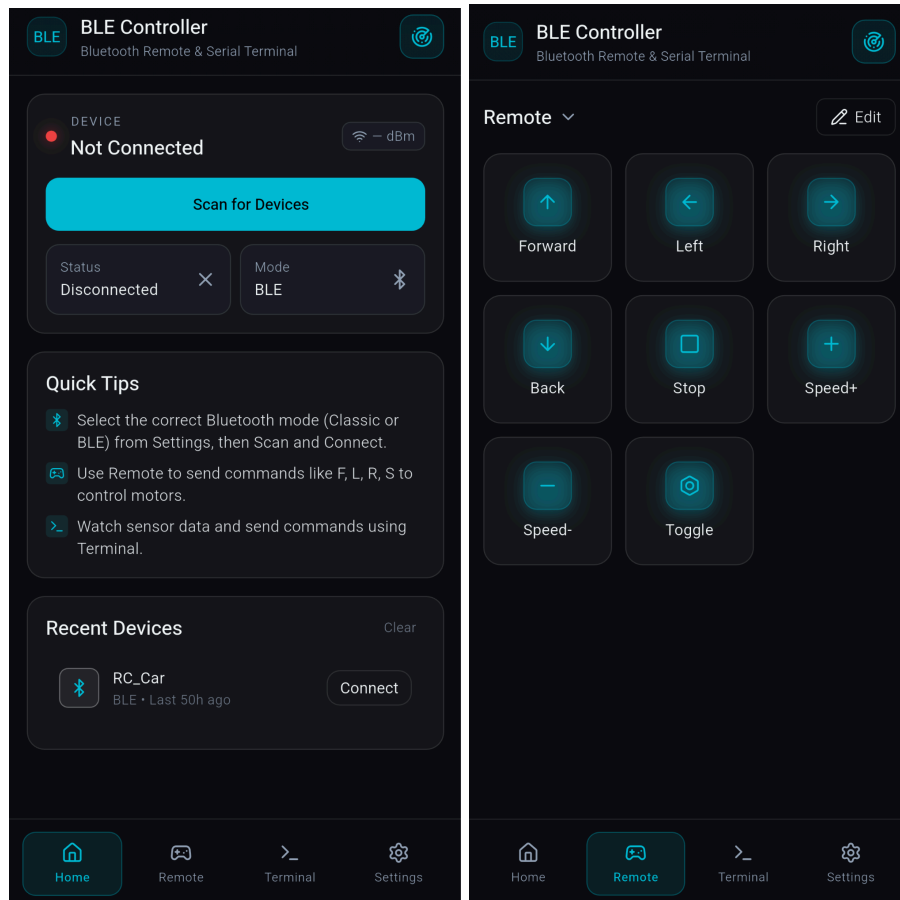
8. Control App

The RC car is controlled using the BLE Controller - Arduino ESP32 by CircuitMagic Inc. Android application. The app allows the user to connect to the device and create custom buttons and choose their command values, in this case the corresponding characteristics.

The controls are:

Command	Action
F	Forward
B	Backward
L	Left
R	Right
S	Stop
+	Increase speed

-	Decrease speed
T	Toggle mode



Driving modes

The core of the vehicle's navigation is a state-managed loop that handles two distinct driving behaviors to compensate for mobile application limitations.

State Tracking:

The system maintains an `activeCmd` variable that persists across loop iterations, ensuring the car doesn't jitter during BLE polling.

One-Press Mode:

In this default state (`holdModeEnabled = false`), a single BLE command triggers a persistent movement state. The car continues its trajectory until an explicit "Stop" ('S') command is received or a new direction is issued.

Hold-to-Move (Watchdog) Logic:

To simulate a "dead-man's switch," the software implements a 300ms CMD_TIMEOUT. If holdModeEnabled is active, the system checks the duration since the last valid command was received (`millis() - lastCmdTime`). If this exceeds the timeout, the `stopMotors()` function is called automatically to prevent runaway behavior.

Mode Synchronization:

Every time the mode is toggled via the 'T' command, the motors are immediately halted to ensure the operator starts from a safe, neutral state.

9. LED Matrix Indications

The built-in LED matrix provides visual feedback for:

- The direction of movement
- Current drive mode
- System status

Displayed icons:

- Forward/Backward arrows
- Left/Right arrows
- Stop symbol
- Mode indications

All bitmap data is stored as const arrays. Because the Arduino LED Matrix library requires writable buffers, the bitmap data is copied from flash memory into a temporary RAM buffer before display. This ensures const-correctness and memory safety.